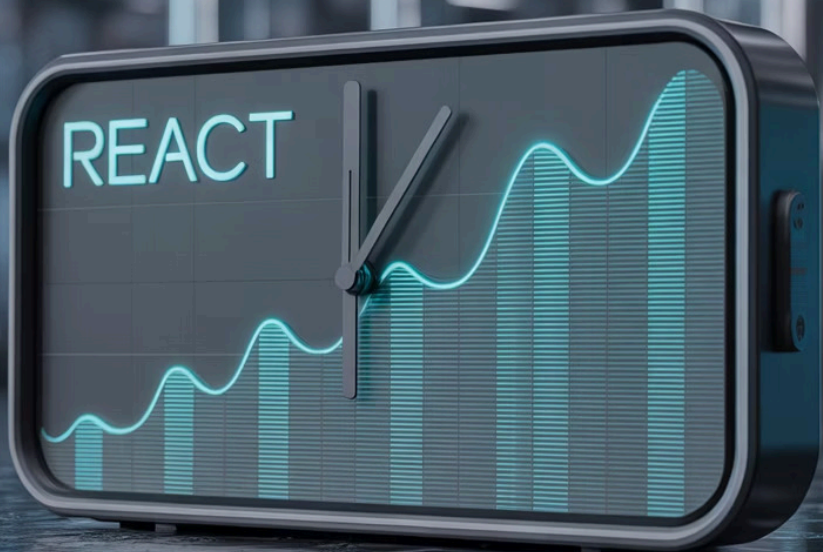




Otimizando Renderizações no React com useCallback

Bem-vindos à nossa jornada pelo hook `useCallback` do React! Nesta apresentação, vamos explorar como esse hook pode ajudar a otimizar suas aplicações React, evitando renderizações desnecessárias e melhorando a performance geral.

O que é o useCallback?

O `useCallback` é um **hook do React** que **memoiza uma função**, garantindo que **a mesma referência de função** seja mantida entre renderizações, a menos que suas dependências mudem.

Em outras palavras, o `useCallback` "memoriza" a função para que ela não seja recriada a cada renderização do componente.



Por que o useCallback é importante?

Recriação de funções

No React, toda vez que um componente renderiza, **todas as funções declaradas dentro dele são recriadas**, gerando novas referências.

Efeitos colaterais

Execução desnecessária de useEffect quando uma função é uma dependência e sua referência muda a cada renderização.

Re-renderizações

Renderizações desnecessárias de componentes filhos que recebem funções como props, mesmo usando React.memo.

Performance

Problemas de performance em listas ou UIs dinâmicas que dependem de funções estáveis para otimização.



Problema comum na prática

Efeitos sendo disparados desnecessariamente

```
useEffect(() => {  
  console.log('Função mudou!');  
  // Alguma lógica importante  
}, [minhaFuncao]);
```

```
function minhaFuncao() {  
  // lógica da função  
}
```

Mesmo que a função faça a mesma coisa, ela é **recriada em cada renderização**, disparando o `useEffect` novamente e potencialmente causando problemas de performance.

Solução: useCallback

Antes (problema)

```
// Criada novamente a cada renderização
const handleClick = () => {
  console.log('Clicado!');
};
```

Depois (solução)

```
// Memoizada - mesma referência entre renderizações
const handleClick = useCallback(() => {
  console.log('Clicado!');
}, []);
```

Com o useCallback, a função **só será recriada se as dependências mudarem**, mantendo a mesma referência entre renderizações.

Isso evita:

- Efeitos colaterais indesejados
- Re-renderizações desnecessárias
- Problemas de performance em componentes otimizados

Diferença entre useMemo e useCallback

Aspecto	useMemo	useCallback
O que memoiza?	O resultado de uma função	A referência da função
Quando usar?	Cálculos pesados que não precisam ser refeitos	Funções passadas como props, dependências de efeitos
Retorno	Um valor (número, objeto, array, etc.)	Uma função
Sintaxe	<code>useMemo(() => valor, [deps])</code>	<code>useCallback(() => função, [deps])</code>

Exemplo prático simples

```
import React, { useState, useCallback } from 'react';

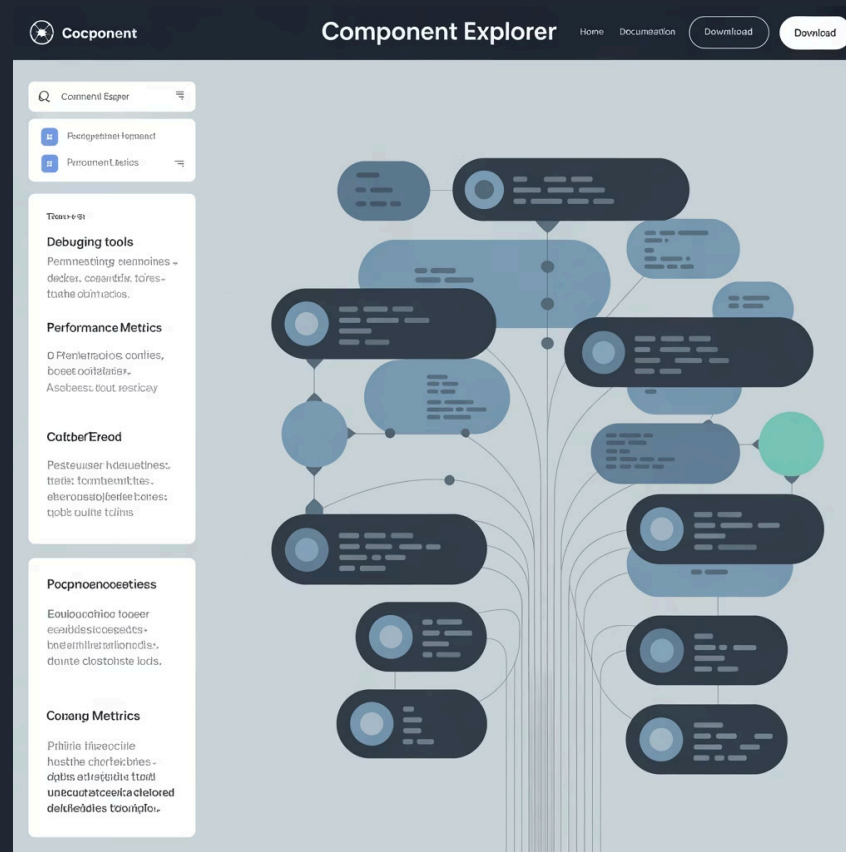
function Pai() {
  const [contador, setContador] = useState(0);

  // Com useCallback – mesma referência entre renderizações
  const handleClick = useCallback(() => {
    console.log('Botão clicado!');
    setContador(c => c + 1);
  }, []);

  return (
    <>
      <button onClick={handleClick}>Contador: {contador}</button>
    </>
  );
}

export default Pai;
```

Sem o `useCallback`, a função `handleClick` seria recriada a cada renderização do componente `Pai`, o que causaria uma re-renderização desnecessária do `ComponenteFilho`, mesmo se ele estiver otimizado com `React.memo`.





Quando usar o useCallback?

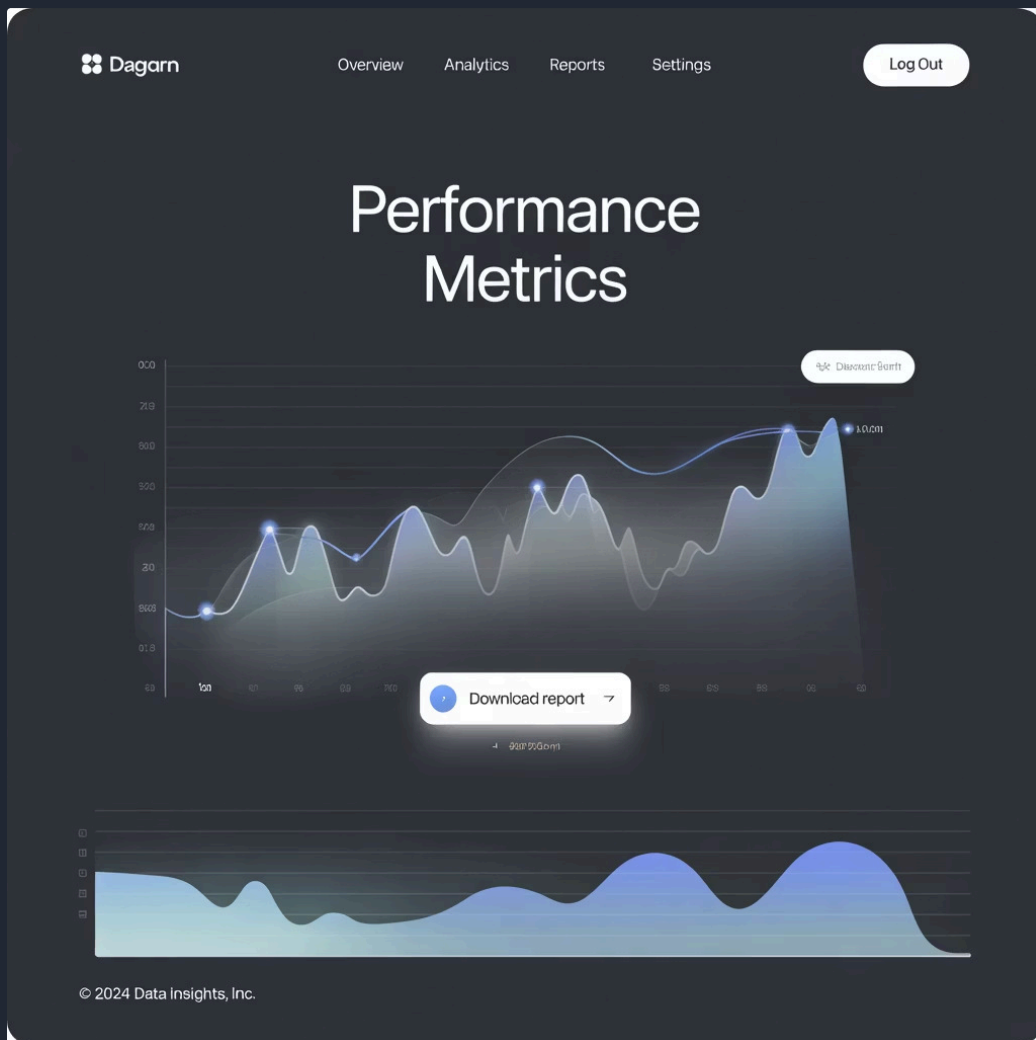
Use quando:

- Você **passa funções como props** para componentes filhos com React.memo
- Está utilizando funções dentro de useEffect ou useLayoutEffect
- Você quer **evitar re-renderizações desnecessárias** por conta de referências diferentes
- Está trabalhando com listas grandes ou interfaces complexas

Evite usar quando:

- A função é pequena e não é passada para componentes filhos
- O uso de useCallback está tornando o código mais difícil de ler sem ganhos reais
- Você está trabalhando com componentes simples que não precisam de otimização
- A função depende de muitos valores que mudam frequentemente

Observação importante sobre memoização



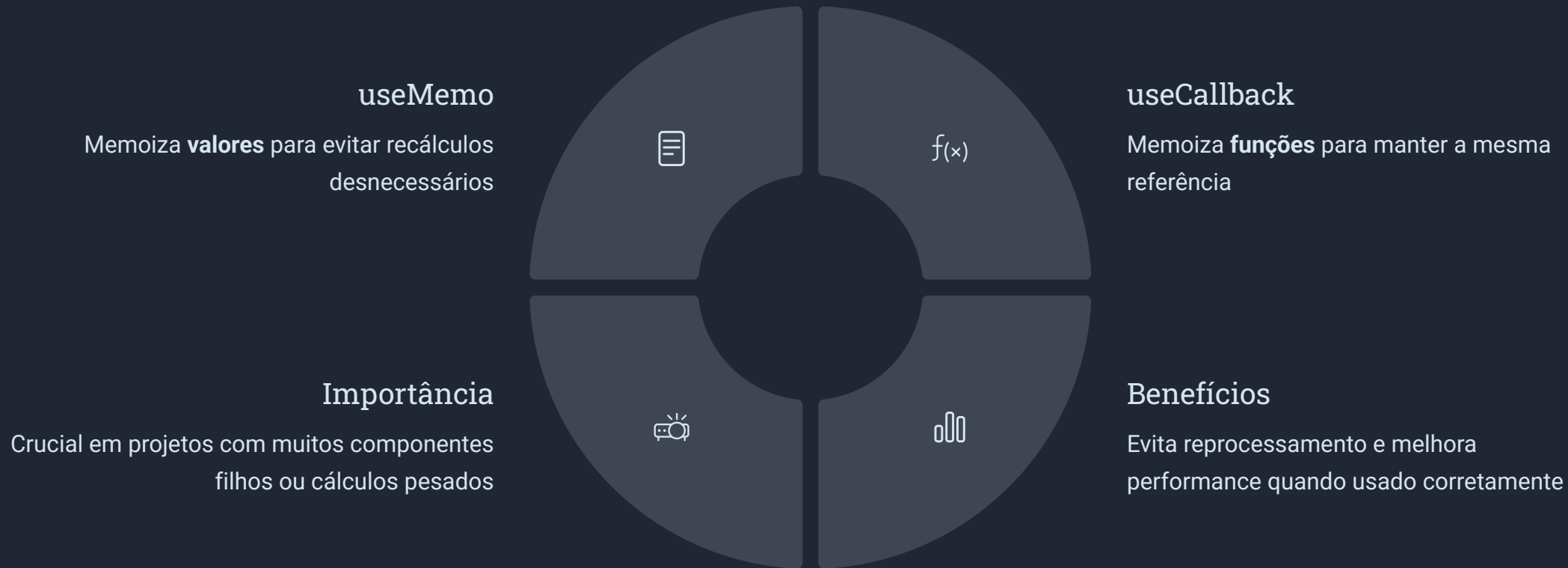
A memoização **não melhora a performance em todos os casos**. Na verdade, pode até piorar se usada indiscriminadamente.

Ela deve ser usada **quando há re-renderizações evitáveis** causadas por mudanças de referência, especialmente em:

- UIs grandes e complexas
- Listas com muitos itens
- Componentes otimizados com React.memo
- Funções usadas como dependências de useEffect

⊗ A otimização prematura pode tornar seu código mais complexo sem benefícios reais!

Recapitulando



Na próxima aula, aplicaremos esses conceitos na prática, criando um relógio digital com atualização por segundo, onde veremos como o `useCallback` pode melhorar significativamente a performance.